

Formal Languages and Compilers

Prof. Breveglieri, Morzenti, Agosta

Written exam: laboratory question

13/02/2024

Time: 60 minutes. Textbooks and notes can be used. Pencil writing is allowed.

Important: Write your name on any additional sheet.

SURNAME (Cognome):

NAME (Nome):

Matricola:or Person Code:

Instructor: ☐ Prof. Breviglieri ☐ Prof. Morzenti ☐ Prof. Agosta

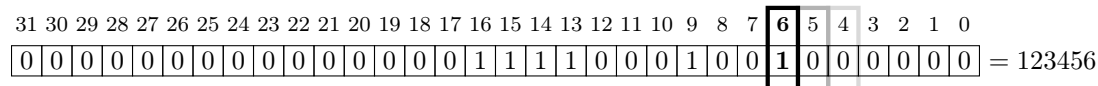
The laboratory question must be answered taking into account the implementation of the Acse compiler given with the exam text.

Modify the specification of the lexical analyser (`flex` input) and the syntactic analyser (`bison` input) and any other source file required to extend the Lance language with the ability to support the **pick** expression operator. This operator takes an array identifier and an expression as arguments:

pick ($\langle array \rangle$, $\langle exp \rangle$)

When it is computed, `pick` searches for the first bit set to 1 in the value of $\langle exp \rangle$, scanning the bits starting from the least significant one. Then it reads the element in $\langle array \rangle$ at the same *index* as the bit found, and it returns the value of the array element as the value of the operator. Bit indices are considered starting from 0 for the least significant bit.

As an example, consider the number 123456. The representation of this number in binary and the numbering of the bits is shown in the following figure.



When scanning the bits from right to left (i.e. from the least significant bit to the most significant) the first 1 we find is on bit 6. As a result, `pick` will read the array element at index 6 and return it.

If there are no bits set to 1 in the value of $\langle exp \rangle$, or if the index of the bit exceeds the bounds of $\langle array \rangle$, the value of the `pick` operator will be 0. Additionally, the modified compiler shall produce a syntax error if $\langle array \rangle$ is not a valid array identifier. No constant folding is required.

The following code snippet exemplifies the functionality of the operator.

```
int a[10], c;
// ... (array initialization omitted) ...
// a = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

c = pick(a, 123456) * 1024;    // The least significant bit set to 1 is bit 6
write(c);                     // Prints a[6] * 1024 = 7168
c = pick(a, c);                // The first bit set to 1 is bit 10, out of bounds!
write(c);                       // Prints 0
write(pick(a, c - 7168));       // The expression is equal to zero, prints 0
```

Tip: The following expression is true if bit n of variable x is set to 1: $(x \gg n) \ \& \ 1$

1. Define the tokens (and the related declarations in **Acse.lex** and **Acse.y**). (2 points)
2. Define the syntactic rules or the modifications required to the existing ones. (3 points)
3. Define the semantic actions needed to implement the required functionality. (20 points)

Solution

Outline The outline of the solution consists in generating assembly instructions equivalent to the following C-like pseudo-code:

```
int res = 0;
if (exp != 0) {
    int bitIndex = 0;
    while (((exp >> bitIndex) & 1) == 0)
        bitIndex++;
    if (bitIndex < size(array))
        res = array[bitIndex];
}
// Result is in 'res'
```

This was the algorithm suggested by the tip in the text; other approaches for computing the index of the least significant bit set to 1 were possible. For example:

```
int temp = exp;
int bitIndex = 0;
while ((temp & 1) == 0 && bitIndex < 32) {
    temp = temp >> 1;
    bitIndex++;
}
if (bitIndex < 32 && bitIndex < size(array))
    res = array[bitIndex];
else
    res = 0;
// Result is in 'res'
```

Both solution outlines are equally valid, and even more variants exist.

In case the expression is equal to zero, or in case the bit index is too large and exceeds the maximum index in the array, the result “returned” by the operator is **zero**, not *the value of the array element at index zero*. Henceforth, the array access must be performed conditionally: a solution which generates code that doesn’t do so is incorrect.

Constant folding When it comes to actually writing the code generation procedure in the semantic actions, the fact that *constant folding is not required* means that there is no need to compute the bit index at compile-time in case $\langle exp \rangle$ is constant (i.e. of type IMMEDIATE). However, it is still possible for $\langle exp \rangle$ to be constant because of the constant folding performed by the other expression-related rules. As a result, **it is an error to not check the expression_type of $\langle exp \rangle$** .

In any case, as arrays are by definition not constant, the resulting expression to be assigned to \$\$ in the rule must be of REGISTER type regardless.

Assignment of the semantic value Additional care must be taken when assigning the semantic value of the operator (i.e. \$\$). Remember that semantic actions are just pieces of C code, and assignments to a variable do not have side-effects affecting code generation. As a consequence, **assigning to \$\$ is not like a return statement of a function that executes at run-time**.

On the other hand, **assigning to \$\$ is a compile-time decision about the location where the result of the expression is located**. This location is encoded in the program, and at runtime it is not possible to change it.

For this reason, solutions containing code similar to this are **incorrect**:

```

t_axe_label *l_ok = newLabel(program);
t_axe_label *l_end = newLabel(program);

gen_subi_instruction(program, REG_0, r_exp, 0);
gen_bne_instruction(program, l_ok, 0);
$$ = create_expression(0, IMMEDIATE);    // useless assignment! incorrect
gen_bt_instruction(program, l_end);
assignLabel(program, l_ok);
int r_res = loadArrayElement(program, ...);
$$ = create_expression(r_res, REGISTER);  // overwrites the assignment above
assignLabel(program, l_end);

```

The first assignment to \$\$ effectively has no effect because it is overwritten by the second one. Indeed, the generation of branch instructions is not conditional at compile-time, but only at runtime. Instead, assignments to \$\$ happen at compile time and do not add any instruction to the output. The final effect is that the final location of the result of the expression is a register that contains the result of the array access if the access is valid, but if the access is not performed the register stays uninitialized.

Note that this reasoning does not apply just to assigning to \$\$ but to assignments to any other variables as well. Therefore the same mistake might have looked like this:

```

t_axe_label *l_end = newLabel(program);

int r_res = gen_load_immediate(program, 0); // useless assignment! incorrect
$$ = create_expression(r_res, REGISTER);

gen_subi_instruction(program, REG_0, r_exp, 0);
gen_beq_instruction(program, l_end, 0);
int r_res = loadArrayElement(program, ...); // overwrites the assignment above
assignLabel(program, l_end);

```

Again there is an assignment statement surrounded or *guarded* by calls to `gen_b...instruction()`, even though the latter does not affect the control flow at *compile time*, just at runtime.

A **correct** solution instead looks like this:

```

t_axe_label *l_ok = newLabel(program);
t_axe_label *l_end = newLabel(program);

int r_res = getNewRegister(program);
$$ = create_expression(r_res, REGISTER);

gen_subi_instruction(program, REG_0, r_exp, 0);
gen_bne_instruction(program, l_ok, 0);
gen_addi_instruction(program, r_res, REG_0, 0);
gen_bt_instruction(program, l_end);
assignLabel(program, l_ok);
int r_tmp = loadArrayElement(program, ...);
gen_addi_instruction(program, r_res, r_tmp, 0);
assignLabel(program, l_end);

```

Notice how the assignment to \$\$ is **not surrounded by `gen_b...instruction()` calls**. The expression's result is always stored in the same register, but the value assigned to this register is changed *at runtime* by the instructions written by `gen_addi_instruction()`.

Even though this is not very relevant for this exercise, of course there is no problem with assigning \$\$ to two different values depending on a *compile-time-evaluated condition*. For instance, if one wishes to also check the expression value at compile time (if possible), the following code snippet (some parts omitted) is correct:

```

exp: PICK LPAR IDENTIFIER COMMA exp RPAR
{
    t_axe_variable *v_array = getVariable(program, $3);
    ...
    if ($5.expression_type == IMMEDIATE) {
        int bitIndex = ...
        if (bitIndex < 32 && bitIndex < v_array->arraySize) {
            int r_res = loadArrayElement(program, ...);
            $$ = create_expression(r_res, REGISTER);
        } else {
            $$ = create_expression(0, IMMEDIATE);
        }
    } else {
        int r_res = getNewRegister(program);
        $$ = create_expression(r_res, REGISTER);
        // Multiple assignments to $$ here do not make sense!
        ...
    }
}
...

```

Notice how the assignments to \$\$ are guarded by normal if statements, which is what makes this code correct.

Complete patch

The solution is shown below in *diff* format. All lines that begin with '+' were added, while the lines that begin with '-' were removed. **It is not required and it is not encouraged to provide a solution in *diff* format to get the maximum grade.**

```

diff --git a/acse/Acse.lex b/acse/Acse.lex
index 35f9b73..ab3090f 100644
--- a/acse/Acse.lex
+++ b/acse/Acse.lex
@@ -93,6 +93,7 @@ ID      [a-zA-Z_][a-zA-Z0-9_]*
"return"      { return RETURN; }
"read"        { return READ; }
"write"       { return WRITE; }
+"pick"       { return PICK; }

{ID}          { yylval.svalue=strdup(yytext); return IDENTIFIER; }
{DIGIT}+      { yylval.intval = atoi( yytext ); }
diff --git a/acse/Acse.y b/acse/Acse.y
index 5e31c71..c77136c 100644
--- a/acse/Acse.y
+++ b/acse/Acse.y
@@ -125,6 +125,7 @@ extern void yyerror(const char*);
%token RETURN
%token READ
%token WRITE
+%token PICK

%token <label> DO
%token <while_stmt> WHILE
@@ -458,7 +459,68 @@ write_statement : WRITE LPAR exp RPAR
    }

;

-exp: NUMBER { $$ = create_expression($1, IMMEDIATE); }
+exp: PICK LPAR IDENTIFIER COMMA exp RPAR
+{
+    // Check if the identifier is a valid array name.
+    t_axe_variable *v_array = getVariable(program, $3);
+    if (!v_array || !v_array->isArray) {

```

```

+         yyerror("first argument to pick must be an array");
+         YYERROR;
+     }
+
+     // Reserve a register to contain the result, initially set to 0
+     int r_res = gen_load_immediate(program, 0);
+     $$ = create_expression(r_res, REGISTER);
+     // Generate code to copy the $5 expression value into a new register.
+     int r_exp = getNewRegister(program);
+     if ($5.expression_type == REGISTER) {
+         gen_add_instruction(program, r_exp, REG_0, $5.value, CG_DIRECT_ALL);
+     } else {
+         gen_addi_instruction(program, r_exp, REG_0, $5.value);
+     }
+     // Generate code which checks if the expression is equal to zero, and in
+     // that case exits immediately. Note that the previous add(i) instruction
+     // already set the Z flag appropriately for checking equality to zero.
+     t_axe_label *l_exit = newLabel(program);
+     gen_beq_instruction(program, l_exit, 0);
+
+     // Generate a do-while loop which searches the first bit set to 1 in the
+     // expression value.
+     // Reserve and generate code to initialize a register which will count
+     // the bit we are currently checking
+     int r_bitIndex = gen_load_immediate(program, 0);
+     // Emit a label for the back-edge of the loop.
+     t_axe_label *l_loop = assignNewLabel(program);
+     // Generate code that checks if (Rexp >> RbitIndex) & 1 == 0 and jumps
+     // out of the loop ("found" label) if the condition is true
+     int r_temp = getNewRegister(program);
+     gen_shr_instruction(program, r_temp, r_exp, r_bitIndex, CG_DIRECT_ALL);
+     gen_andbi_instruction(program, r_temp, r_temp, 1);
+     t_axe_label *l_found = newLabel(program);
+     gen_bne_instruction(program, l_found, 0);
+     // Generate code to increment the bit index and jump back if the
+     // condition was false.
+     gen_addi_instruction(program, r_bitIndex, r_bitIndex, 1);
+     gen_bt_instruction(program, l_loop, 0);
+     assignLabel(program, l_found);
+
+     // After the loop, RbitIndex contains the index used to access the array.
+     // Generate code which checks if it's not too large, and in that case
+     // exits leaving the default return value of zero.
+     gen_subi_instruction(program, REG_0, r_bitIndex, v_array->arraySize);
+     gen_bge_instruction(program, l_exit, 0);
+     // Generate the array access and the copy of the value to the result reg.
+     int r_array = loadArrayElement(program, $3,
+         create_expression(r_bitIndex, REGISTER));
+     gen_add_instruction(program, r_res, REG_0, r_array, CG_DIRECT_ALL);
+
+     // Assign the exit label
+     assignLabel(program, l_exit);
+
+     // Free the identifier string
+     free($3);
+ }
+ | NUMBER      { $$ = create_expression ($1, IMMEDIATE); }
+ | IDENTIFIER  {
+
+         int location;

```

```

diff --git a/tests/pick/pick.src b/tests/pick/pick.src
new file mode 100644
index 0000000..d6cf364
--- /dev/null
+++ b/tests/pick/pick.src
@@ -0,0 +1,26 @@
+int a[10], i, c;
+
+
+i = 0;

```

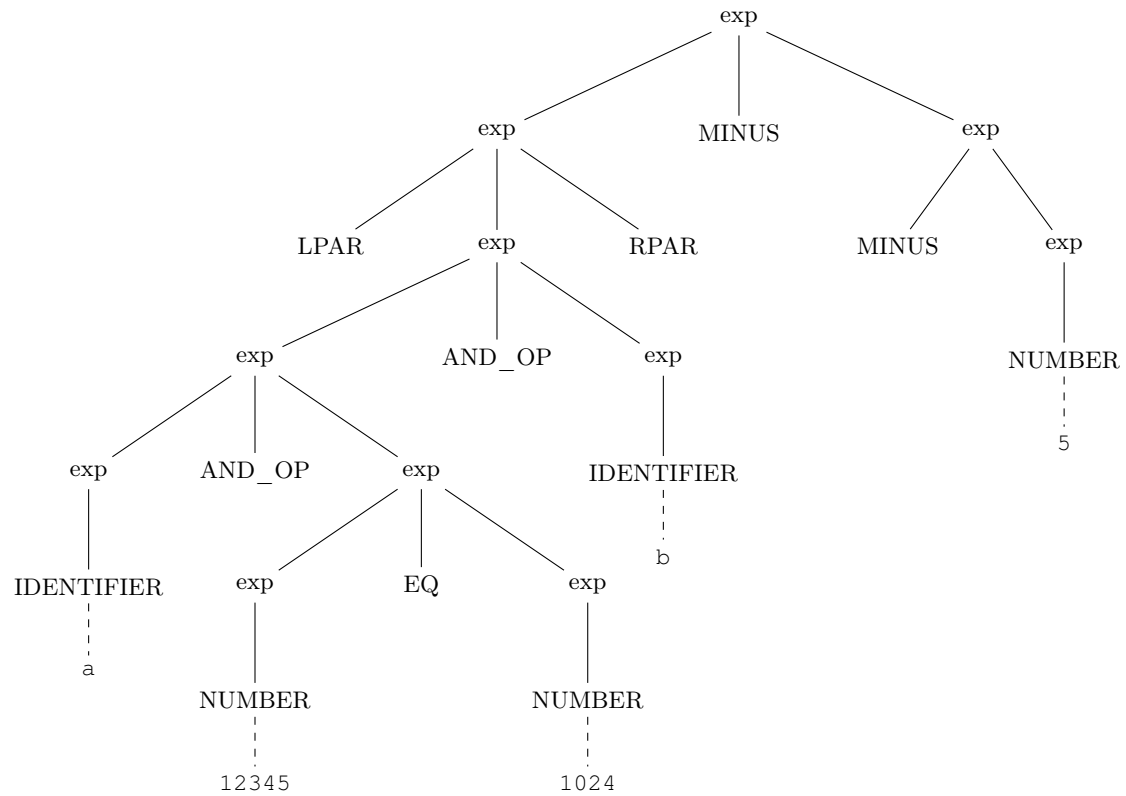
```
+while (i < 10) {  
+  a[i] = i+1;  
+  i = i+1;  
+}  
+  
+write(pick(a, 0));  
+write(pick(a, 1));  
+write(pick(a, 2));  
+write(pick(a, 4));  
+write(pick(a, 8));  
+write(pick(a, 16));  
+write(pick(a, 32));  
+write(pick(a, 64));  
+write(pick(a, 128));  
+write(pick(a, 256));  
+write(pick(a, 512));  
+write(pick(a, 1024));  
+  
+c = pick(a, 123456) * 1024;  
+write(c); // Prints a[6] * 1024 = 7168  
+c = pick(a, c);  
+write(c); // Prints 0  
+write(pick(a, c - 7168)); // Prints 0
```

4. Given the following Lance code snippet:

```
(a & 12345 == 1024 & b) --5
```

write down the syntactic tree generated during the parsing with the Bison grammar described in Acse.y *starting from the exp nonterminal*. (5 points)

Solution



5. (**Bonus**) Briefly explain in plain English words how to modify the solution given to the questions about the `pick` operator in order to also reset the bit found equal to 1 to 0.

Notice that this extension requires modifying the syntax of `pick` to take a *scalar variable identifier* instead of an *expression* as second argument. Why?